

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Anul 2011 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Titularizare Varianta 2 (13 iulie 2011)	2
SUBIECTUL I (30 de puncte)	2
1. Conexitatea grafurilor neorientate	2
2. Programare: Numere prime (tprim)	2
a) Metodă și pseudocod pentru al n-lea număr prim	2
b) Subprogramul tprim	3
c) Program care verifică dacă p se află printre primele n numere prime	3
SUBIECTUL al II-lea (30 de puncte)	5
1. Structura sistemelor de calcul: Dispozitive periferice	5
2. Structuri de date: Înregistrare (produs)	5
3. Algoritm eficient: Drum de sumă maximă în matrice pătratică (DP)	6
SUBIECTUL al III-lea (30 de puncte)	8
Test scris sumativ - Informatică, clasa a IX-a	8

I. Rezolvare Titularizare Varianta 2 (13 iulie 2011)

[Subiect PDF](file:

SUBIECTUL I (30 de puncte)

1. Conexitatea grafurilor neorientate

• Definiții:

- **Graf neorientat:** O pereche $G = (V, E)$ de noduri și muchii.
- **Adiacență:** Relația dintre două noduri conectate printr-o muchie.
- **Lanț:** O succesiune de noduri adiacente.
- **Conexitate:** Un graf este conex dacă între oricare două noduri există cel puțin un lanț.
- **Componentă conexă:** Un subgraf conex maximal.

- **Exemplu și contraexemplu:** Graful cu muchiile $(1,2)$, $(2,3)$, $(3,4)$ este conex, deoarece orice două noduri pot fi unite printr-un lanț. Graful cu muchiile $(1,2)$ și $(3,4)$ nu este conex, deoarece nu există lanț între nodurile 1 și 3.

• Aplicații practice:

- **Rețele sociale:** Determinarea grupurilor de prieteni interconectați.
- **Infrastructură:** Verificarea dacă toate orașele sunt legate rutier/feroviar.

- **Algoritm de determinare:** Se pornesc parcurgeri BFS/DFS din fiecare nod nevizitat. Toate nodurile atinse într-o astfel de parcurgere formează o componentă conexă; numărul parcurgerilor pornite este numărul componentelor conexe.

Pseudocod:

```

citeste n, m
pentru fiecare muchie (x, y):
    adauga y in lista[x]
    adauga x in lista[y]
pentru i <- 1..n:
    vizitat[i] <- fals
nrComp <- 0
pentru i <- 1..n:
    daca vizitat[i] = fals atunci
        nrComp <- nrComp + 1
        DFS(i, nrComp)

procedura DFS(x, nrComp):
    vizitat[x] <- adevarat
    componenta[x] <- nrComp
    pentru fiecare vecin y al lui x:
        daca vizitat[y] = fals atunci DFS(y, nrComp)
    
```

—

2. Programare: Numere prime (tprim)

a) Metodă și pseudocod pentru al n-lea număr prim

Descriere: Numărăm valorile prime întâlnite în ordine crescătoare, pornind de la 2. Pentru fiecare candidat x , testăm dacă are divizori între 2 și $\sqrt{x}$. Când contorul numerelor prime găsite devine egal cu n , valoarea curentă este al n -lea termen din șirul numerelor prime.

Pseudocod:

```

citeste n
cnt <- 0
x <- 1
cat timp cnt < n executa
    x <- x + 1
    prim <- adevarat
    daca x < 2 atunci prim <- fals
    d <- 2
    cat timp d * d <= x si prim = adevarat executa
        daca x mod d = 0 atunci prim <- fals
        d <- d + 1
    daca prim = adevarat atunci cnt <- cnt + 1
scrie x

```

b) Subprogramul tprim

Subprogramul primește un număr natural nenul n și returnează al n -lea număr prim. Pentru claritate, funcția auxiliară `isPrime` testează primalitatea.

Soluție C++:

```

#include <iostream>
using namespace std;

bool isPrime(long long val) {
    if (val < 2) return false;
    for (long long d = 2; d * d <= val; ++d) {
        if (val % d == 0) return false;
    }
    return true;
}

int tprim(int n) {
    int count = 0;
    int val = 2;
    while (true) {
        if (isPrime(val)) {
            count++;
            if (count == n) return val;
        }
        val++;
    }
}

int main() {
    int n;
    cin >> n;
    cout << tprim(n);
    return 0;
}

```

c) Program care verifică dacă p se află printre primele n numere prime

Metoda cerută folosește apelul `tprim(n)`: dacă p este prim și $p \leq \text{tprim}(n)$, atunci p apare în primele n numere prime. Dacă p este compus sau este mai mare decât al n -lea prim, răspunsul este NU.

Soluție C++:

```
#include <iostream>
using namespace std;

bool isPrime(long long val) {
    if (val < 2) return false;
    for (long long d = 2; d * d <= val; ++d) {
        if (val % d == 0) return false;
    }
    return true;
}

int tprim(int n) {
    int count = 0;
    int val = 2;
    while (true) {
        if (isPrime(val)) {
            count++;
            if (count == n) return val;
        }
        val++;
    }
}

int main() {
    long long p;
    int n;
    if (!(cin >> p >> n)) return 0;

    long long limit = tprim(n);
    if (p <= limit && isPrime(p)) {
        cout << "DA\n";
    } else {
        cout << "NU\n";
    }
    return 0;
}
```

Soluție Pascal:

```
program VerificarePrime;
var
    p: int64;
    n: integer;
    limit: int64;

function isPrime(val: int64): boolean;
var d: int64;
begin
    if val < 2 then
        begin
            isPrime := false;
            exit;
        end;
    d := 2;
    while d * d <= val do
        begin
            if val mod d = 0 then
```

```

begin
    isPrime := false;
    exit;
end;
d := d + 1;
end;
isPrime := true;
end;

function tprim(n_val: integer): integer;
var
    count, val: integer;
begin
    count := 0;
    val := 2;
    while true do
        begin
            if isPrime(val) then
                begin
                    count := count + 1;
                    if count = n_val then
                        begin
                            tprim := val;
                            exit;
                        end;
                    end;
                    val := val + 1;
                end;
            end;
        end;
    end;

begin
    readln(p, n);
    limit := tprim(n);
    if (p <= limit) and isPrime(p) then
        writeln('DA')
    else
        writeln('NU');
    end.
end.

```

SUBIECTUL al II-lea (30 de puncte)

1. Structura sistemelor de calcul: Dispozitive periferice

- **Rol:** Interfață I/O pentru comunicarea datelor cu UCP.
- **Clasificare:**
 - **Intrare:** Tastatură, Mouse.
 - **Ieșire:** Monitor, Imprimantă.
 - **Stocare/Mixte:** Placă de rețea.
- **Parametri:** Rezoluție DPI monitor, polling rate mouse, viteza de printare (PPM).

2. Structuri de date: Înregistrare (produs)

- **C++:**

```
struct produs {
    char denumire[100];
    double pret;
};
produs x[2000];
// Afișare preț pe poziția 10
cout << x[9].pret; // 0-indexed sau x[10].pret dacă e 1-indexed
```

• **Pascal:**

```
type
    produs = record
        denumire: string;
        pret: real;
    end;
var
    x: array[1..2000] of produs;
// Afișare preț pe poziția 10
writeln(x[10].pret);
```

—

3. Algoritm eficient: Drum de sumă maximă în matrice pătratică (DP)

Metoda folosită: Programarea dinamică. Notăm $dp[i][j]$ suma maximă a unui șir valid care se termină în elementul de pe linia i , coloana j . Pentru prima linie, $dp[1][j] = A[1][j]$. Pentru liniile următoare, elementul curent poate veni doar din coloana j , $j-1$ sau $j+1$ de pe linia precedentă, deci:

$$dp[i][j] = A[i][j] + \max(dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1])$$

se păstrează doar termenii care există în matrice. Răspunsul este maximul de pe ultima linie a tabloului dp .

Eficiență: Fiecare element al matricei este prelucrat o singură dată, iar pentru fiecare se fac cel mult trei comparații. Complexitatea este $O(n^2)$ timp și $O(n^2)$ memorie în varianta clară de mai jos; memoria poate fi redusă la $O(n)$ păstrând numai linia precedentă.

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    ifstream fin("NR.TXT");
    int n;
    if (!(fin >> n)) return 0;
    vector<vector<long long>> A(n, vector<long long>(n));
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            fin >> A[i][j];
        }
    }
    fin.close();

    vector<vector<long long>> dp(n, vector<long long>(n, 0));
```

```

for (int j = 0; j < n; ++j) {
    dp[0][j] = A[0][j];
}

for (int i = 1; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
        long long prev_max = dp[i-1][j];
        if (j > 0) prev_max = max(prev_max, dp[i-1][j-1]);
        if (j < n - 1) prev_max = max(prev_max, dp[i-1][j+1]);
        dp[i][j] = A[i][j] + prev_max;
    }
}

long long ans = dp[n-1][0];
for (int j = 1; j < n; ++j) {
    ans = max(ans, dp[n-1][j]);
}

cout << ans << endl;
return 0;
}

```

Soluție Pascal:

```

program MaxPathSum;
var
    fin: text;
    n, i, j: integer;
    A: array[1..100, 1..100] of integer;
    dp: array[1..100, 1..100] of int64;
    prev_max, ans: int64;
begin
    assign(fin, 'NR.TXT'); reset(fin);
    read(fin, n);
    for i := 1 to n do
        for j := 1 to n do
            read(fin, A[i, j]);
    close(fin);

    for j := 1 to n do dp[1, j] := A[1, j];

    for i := 2 to n do
        begin
            for j := 1 to n do
                begin
                    prev_max := dp[i - 1, j];
                    if j > 1 then
                        begin
                            if dp[i - 1, j - 1] > prev_max then prev_max := dp[i - 1, j - 1];
                        end;
                    if j < n then
                        begin
                            if dp[i - 1, j + 1] > prev_max then prev_max := dp[i - 1, j + 1];
                        end;
                    dp[i, j] := A[i, j] + prev_max;
                end;
            end;
        end;
end;

```

```
ans := dp[n, 1];
for j := 2 to n do
begin
  if dp[n, j] > ans then ans := dp[n, j];
end;
writeln(ans);
end.
```

SUBIECTUL al III-lea (30 de puncte)

Test scris sumativ - Informatică, clasa a IX-a

Capitole: algoritmi elementari, structuri de control, tablouri unidimensionale. **Timp de lucru:** 50 de minute. Se acordă 90 de puncte și 10 puncte din oficiu.

Item	Enunț	Răspuns / barem
Pereche 1 (10p)	Asociați operația cu rolul ei: $n \bmod 10$, $n \div 10$, $i++$.	$n \bmod 10$ - ultima cifră; $n \div 10$ - elimină ultima cifră; $i++$ - incrementare.
Pereche 2 (10p)	Asociați structura cu situația: if, while, for.	if - decizie; while - repetare cu număr necunoscut de pași; for - număr cunoscut de pași.
Răspuns scurt 1 (10p)	Ce afișează algoritmul care însumează cifrele lui 123?	6.
Completare 2 (10p)	Într-un vector v, elementul de pe poziția i se notează [spațiu liber].	$v[i]$ sau $v[i]$ în C/C++, $v[i]/v[i]$ după convenția predată.
Întrebare structurată (20p)	Pentru vectorul 4 7 2 9: precizați maximul, poziția sa și descrieți algoritmul.	Maxim 9, poziția 4; parcurgere liniară cu variabilă max.
Problemă (30p)	Scrieți un algoritm care citește n și n valori și afișează câte sunt pare.	Citire corectă 5p, parcurgere 10p, test paritate 10p, afișare 5p.